

[TGLMR19,CRYPTO] AMF: Content Moderation for Metadata-Private E2EE

SUMMARY: This paper propose an asymmetric message franking scheme that can work in metadata private messaging system.

Observation: In fact, this paper construct a deniable designated-two-verifier signature scheme. The biggest contribution of this paper is the analysis of deniability.

1 Setting & Goals

1.1 Setting

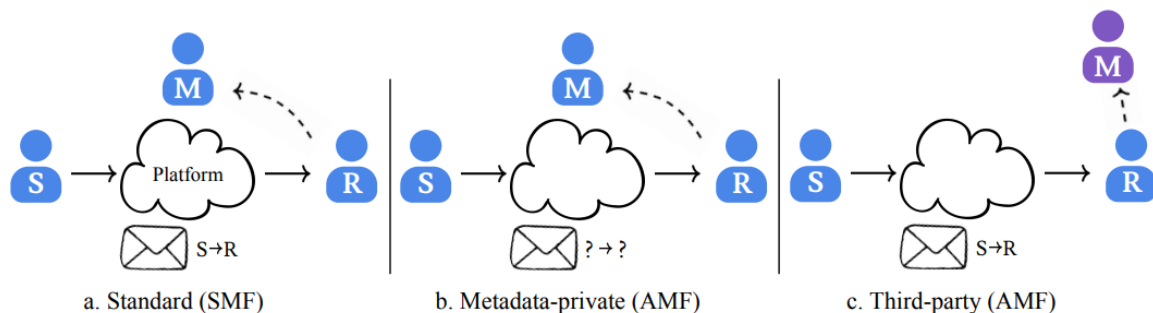


Fig. 1: Overview of different settings for content moderation of messaging. The solid arrow denotes sending a message across the platform and the dashed arrow denotes reporting a message to the moderator. In the standard setting, messages sent across the platform are associated with sender and receiver identities and the platform is the moderator. In the metadata-private setting, the associated sender and receiver identities of messages are hidden from the platform, and by extension, the moderator. In the third-party setting, the moderator is separate from the platform, and thus also cannot associate sender and receiver identities to messages. Our AMF primitive targets the latter two settings.

1.2 Security Goals

There are two parts of security that AMF requires, i.e., accountability and deniability.

Accountability

- Receiver binding
- Sender binding
- Unforgeability

Deniability

- Universal deniability
 - This property is implemented by some kind of indistinguishability. When the signature is indistinguishable to the users except the verifier, a forger can forge valid signature with random values.



Alg.	Security notion	How to prove first clause? ($pk_s = g^t \vee J = g^u$)	How to prove second clause? ($((J = (pk_j)^v \wedge E_J = g^v) \vee R = g^w)$)	Verify?	Judge?
Frank	Correctness	$\alpha \leftarrow \mathbb{Z}_p; J \leftarrow (pk_j)^\alpha; t = sk_s$	$\beta \leftarrow \mathbb{Z}_p; R \leftarrow (pk_r)^\beta; v = \alpha$	✓	✓
Forge	Univ. den.	$\gamma \leftarrow \mathbb{Z}_p; J \leftarrow g^\gamma; u = \gamma$	$\delta \leftarrow \mathbb{Z}_p; R \leftarrow g^\delta; w = \delta$	×	×
RForge	R. comp. den.	$\gamma \leftarrow \mathbb{Z}_p; J \leftarrow g^\gamma; u = \gamma$	$\beta \leftarrow \mathbb{Z}_p; R \leftarrow (pk_r)^\beta; w = \beta \cdot sk_r$	✓	×
JForge	J. comp. den.	$\alpha \leftarrow \mathbb{Z}_p; J \leftarrow (pk_j)^\alpha; u = \alpha \cdot sk_j$	$\beta \leftarrow \mathbb{Z}_p; R \leftarrow (pk_r)^\beta; v = \alpha$	✓	✓

$\mathcal{R} = \{((t, u, v, w), (g, pk_s, pk_j, J, R, E_J)) : (pk_s = g^t \vee J = g^u) \wedge ((J = (pk_j)^v \wedge E_J = g^v) \vee R = g^w)\}$					
<u>Frank(sk_s, pk_r, pk_j, msg):</u>		<u>RForge(pk_s, sk_r, pk_j, msg):</u>		<u>KeyGen:</u>	
$(\alpha, \beta) \leftarrow \mathbb{Z}_p^2$ $J \leftarrow (pk_j)^\alpha$ $R \leftarrow (pk_r)^\beta$ $E_J \leftarrow g^\alpha$ $E_R \leftarrow g^\beta$ $x \leftarrow (sk_s, \perp, \alpha, \perp)$ $y \leftarrow (g, pk_s, pk_j, J, R, E_J)$ $\pi \leftarrow \text{SPoK}^{\mathcal{R}}.\text{prove}(msg, x, y)$ return (π, J, R, E_J, E_R)		$(\alpha, \beta, \gamma) \leftarrow \mathbb{Z}_p^3$ $J \leftarrow g^\gamma$ $R \leftarrow (pk_r)^\beta$ $E_J \leftarrow g^\alpha$ $E_R \leftarrow g^\beta$ $x = (\perp, \gamma, \perp, \beta \cdot sk_r)$ $y \leftarrow (g, pk_s, pk_j, J, R, E_J)$ $\pi \leftarrow \text{SPoK}^{\mathcal{R}}.\text{prove}(msg, x, y)$ return (π, J, R, E_J, E_R)		return PKKeyGen <u>Verify($pk_s, sk_r, pk_j, msg, \sigma$):</u> $(\pi, J, R, E_J, E_R) \leftarrow \sigma$ $y \leftarrow (g, pk_s, pk_j, J, R, E_J)$ $b_1 \leftarrow R = E_R^{sk_r}$ $b_2 \leftarrow \text{SPoK}^{\mathcal{R}}.\text{verify}(msg, \pi, y)$ return $b_1 \wedge b_2$	
<u>Forge(pk_s, pk_r, pk_j, msg):</u>		<u>JForge(pk_s, pk_r, sk_j, msg):</u>		<u>Judge($pk_s, pk_r, sk_j, msg, \sigma$):</u>	
$(\alpha, \beta, \gamma, \delta) \leftarrow \mathbb{Z}_p^4$ $J \leftarrow g^\gamma$ $R \leftarrow g^\delta$ $E_J \leftarrow g^\alpha$ $E_R \leftarrow g^\beta$ $x \leftarrow (\perp, \gamma, \perp, \delta)$ $y \leftarrow (g, pk_s, pk_j, J, R, E_J)$ $\pi \leftarrow \text{SPoK}^{\mathcal{R}}.\text{prove}(msg, x, y)$ return (π, J, R, E_J, E_R)		$(\alpha, \beta) \leftarrow \mathbb{Z}_p^2$ $J \leftarrow (pk_j)^\alpha$ $R \leftarrow (pk_r)^\beta$ $E_J \leftarrow g^\alpha$ $E_R \leftarrow g^\beta$ $x \leftarrow (\perp, \alpha \cdot sk_j, \alpha, \perp)$ $y \leftarrow (g, pk_s, pk_j, J, R, E_J)$ $\pi \leftarrow \text{SPoK}^{\mathcal{R}}.\text{prove}(msg, x, y)$ return (π, J, R, E_J, E_R)		$(\pi, J, R, E_J, E_R) \leftarrow \sigma$ $y \leftarrow (g, pk_s, pk_j, J, R, E_J)$ $b_1 \leftarrow J = E_J^{sk_j}$ $b_2 \leftarrow \text{SPoK}^{\mathcal{R}}.\text{verify}(msg, \pi, y)$ return $b_1 \wedge b_2$	

Fig. 5: Algorithms for our deniable AMF scheme. The relation $\mathcal{R}$ defining our SPoK is depicted at the top.